2) Що говорити на захисті: поетапна розповідь по коду

Ось готовий текст, який можна майже прямо читати.

Загальний вступ

У цій лабораторній роботі я реалізував задачу фільтрації числового списку в мові Erlang чотирма різними способами. За умовою мого варіанту потрібно було видалити зі списку всі числа, що знаходяться в діапазоні від 0 до 10 включно. У результаті в новому списку повинні залишитися тільки ті елементи, які менші за 0 або більші за 10.

Модуль і експорт функцій

На початку програми я оголошую модуль `lab8_var16`. Далі через `export` вказую функції `remove1/1`, `remove2/1`, `remove3/1` і `remove4/1`, щоб їх можна було викликати із зовнішнього середовища, тобто з Erlang shell. Усі ці функції приймають один аргумент, а саме список чисел.

Перша версія, через охоронний вираз і співставлення зі зразком

Функція `remove1/1` реалізує розв'язок за допомогою pattern matching і guard. Список розбивається на голову і хвіст у вигляді `[X|Xs]`, де `X` це поточний елемент, а `Xs` це решта списку. Далі використовується охоронний вираз `when X < 0; X > 10`. Якщо умова істинна, тобто число не належить діапазону від 0 до 10, тоді цей елемент додається до нового списку. Якщо умова не виконується, тоді спрацьовує наступне рівняння функції, і цей елемент пропускається. Базовий випадок `remove1([]) -> []` означає, що якщо список порожній, то результатом теж є порожній список. Таким чином функція рекурсивно переглядає всі елементи і залишає тільки потрібні.

Друга версія, через case

Функція `remove2/1` також працює рекурсивно, але перевірка умови виконується через `case`. Спочатку список так само ділиться на голову і хвіст. Потім обчислюється вираз `(X < 0) orelse (X > 10)`. Якщо результат `true`, елемент додається до нового списку. Якщо `false`, він пропускається. Далі функція знову викликає сама себе для хвоста списку. Тобто логіка така сама, як і в першій функції, але спосіб запису інший. Тут замість guard використовується конструкція `case`.

Третя версія, через lists:filter/2

Функція `remove3/1` є коротшою і використовує стандартну вбудовану функцію `lists:filter/2`. У неї передається анонімна функція `fun(X) -> (X < 0) orelse (X > 10) end` і сам список. `lists:filter/2` проходить по всіх елементах списку і залишає тільки ті, для яких анонімна функція повертає `true`. Тобто тут фільтрація виконується вже готовим інструментом мови, а не вручну через рекурсію.

Четверта версія, через генератор списку

Функція `remove4/1` використовує генератор списку. Запис `[X || X <- List, (X < 0) orelse (X > 10)]` означає, що зі списку `List` послідовно береться кожен елемент `X`, і в

новий список потрапляють тільки ті значення, для яких виконується умова `(X < 0) orelse (X > 10)`. Це один із найзручніших і найкоротших способів реалізації фільтрації в Erlang.

Пояснення логіки задачі

Суть задачі полягає в тому, щоб прибрати зі списку всі числа від 0 до 10 включно. Отже, якщо елемент дорівнює 0, 1, 2 і так далі до 10, він не повинен увійти до результату. Якщо елемент менший за 0 або більший за 10, він зберігається. Наприклад, для списку `[5,-2,11,0,7,15,3,-8]` результатом буде `[-2,11,15,-8]`, тому що числа `5, 0, 7, 3` належать діапазону від 0 до 10 і видаляються.

Чому реалізовано чотири функції

Чотири функції потрібні тому, що за умовою лабораторної роботи одну й ту саму задачу треба показати різними способами. Перша функція демонструє використання охоронних виразів і співставлення зі зразком. Друга показує використання `case`. Третя демонструє роботу з вбудованою функцією `lists:filter/2`. Четверта показує використання генератора списку. Усі вони дають однаковий результат, але записані різними засобами мови Erlang.

Коротко по кожній функції для усної відповіді

`remove1/1` це рекурсивна обробка списку через pattern matching і guard.
`remove2/1` це рекурсивна обробка списку через `case`.
`remove3/1` це фільтрація за допомогою готової функції `lists:filter/2`.
`remove4/1` це фільтрація через генератор списку.